DSA5208: Scalable Distributed Computing for Data Science

Lecture 7: Concepts of Parallel Computing

Anthony Chen

Department of Mathematics, National University of Singapore

AY2026/2027, Semester 1

1 Parallel Computing
2 Computer Architectures
3 Models of Parallel Computing
4 Parallel Computing Frameworks
5 MPI
6 Project 3

What is Parallel Computing?

Parallel computing is a type of computation where many calculations or processes are carried out simultaneously.

- It is defined in contrast to serial computing where only one calculation is happening
- Parallelism happens at all different levels of a computer
- Parallelism is not suitable for all types of problems

Serial vs Parallel Computing

In contrast to serial computing, parallel computing:

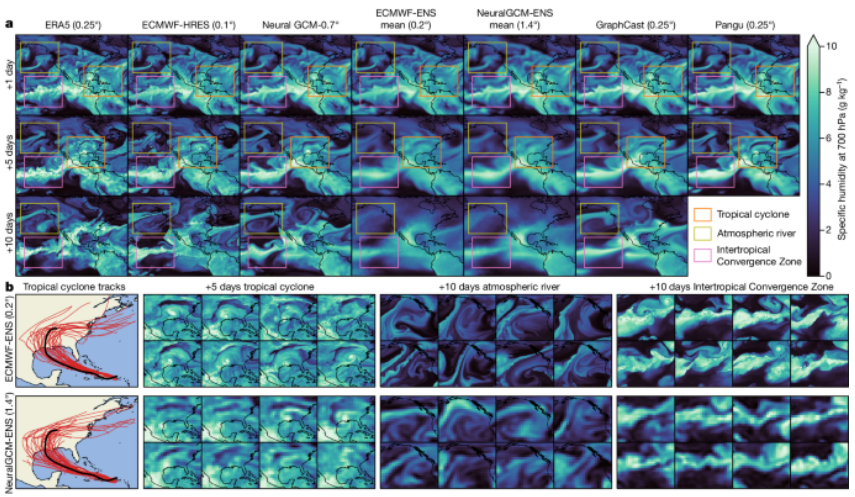
- Is harder to program
- Likewise, harder to debug
- Introduces additional runtime overhead

So why do things in parallel?

Applications of Parallel Computing

Many modern day applications of parallel computing involve very large problems.

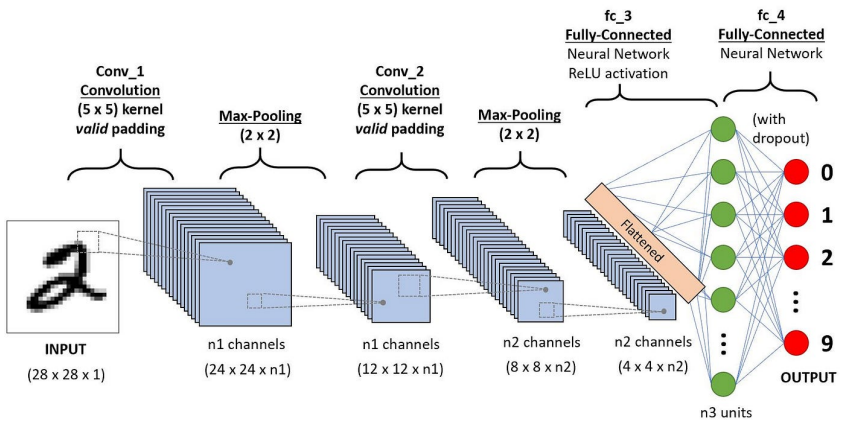
- Computational fluid dynamics
- Weather and climate models



Applications of Parallel Computing

Why do things in parallel then? Many modern day applications of parallel computing involve very large problems.

- Optimization
- Machine Learning



Applications of Parallel Computing

Why do things in parallel then? Many modern day applications of parallel computing involve very large problems.

- Computational biology
- Economics
- Finance
- Fusion
- Astrophysics

Levels of Parallel Computing

Parallel computing occurs at all levels of a computer:

- Bit level parallelism: a processor may operate on multiple bits of data simultaneously

Levels of Parallel Computing

Parallel computing occurs at all levels of a computer:

- Bit level parallelism: a processor may operate on multiple bits of data simultaneously
- Instruction level parallelism: a processor may operate on multiple instructions simultaneously

Levels of Parallel Computing

Parallel computing occurs at all levels of a computer:

- Bit level parallelism: a processor may operate on multiple bits of data simultaneously
- Instruction level parallelism: a processor may operate on multiple instructions simultaneously
- Processor level parallelism: multiple processors may operate simultaneously

Levels of Parallel Computing

Parallel computing occurs at all levels of a computer:

- Bit level parallelism: a processor may operate on multiple bits of data simultaneously
- Instruction level parallelism: a processor may operate on multiple instructions simultaneously
- Processor level parallelism: multiple processors may operate simultaneously
- Node level parallelism: multiple nodes may operate simultaneously

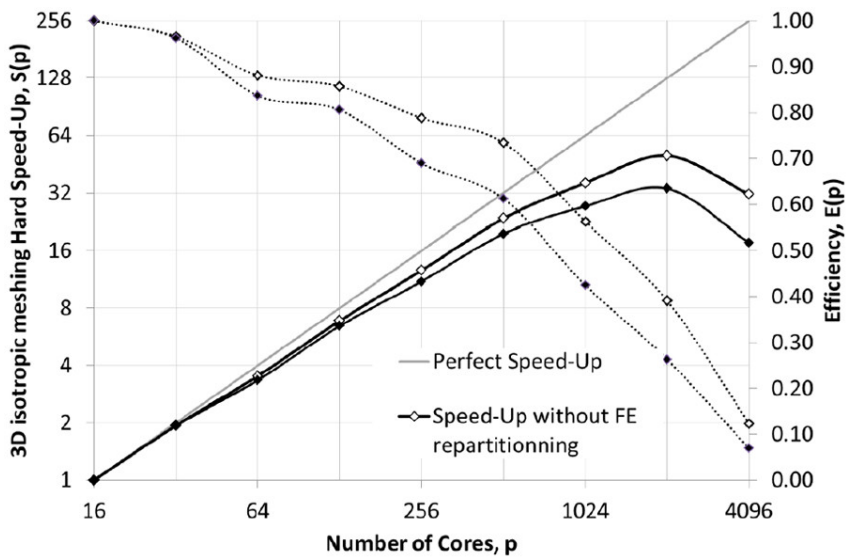
We will focus on processor and node level parallelism.

Measuring Parallel Speedup

How much faster can a parallel program be than a serial program? We have several ways of measuring this. Let T_s be the time it takes for a serial program to run and $T_p(n)$ be the time it takes for a parallel program to run. Then:

- The parallel speedup is $T_s/T_p(n)$
- The parallel efficiency is $T_s/(nT_p(n))$

For perfect parallelism, the speedup is equal to n and the efficiency is 1. Unfortunately, this is not possible.



Amdahl's Law

How much speedup is possible? There are several limitations on speedup, including:

- Portions of the code which are not suited to parallelization
- Added overhead from parallelization
- Communication/synchronization between processes

Amdahl's Law lets us calculate the theoretical maximum speedup possible. Let P be the portion of a program that can be parallelized and $S = 1 - P$ be the serial portion. Then, the theoretical speedup with n processes is

$$\text{Speedup} = \frac{1}{S + P/n} \tag{1}$$

As $n \to \infty$, then the speedup is limited to $1/S$.

Strong and Weak Scaling

Everything previously discussed has assumed a fixed problem size N. What if instead, you increased the problem size as you increased the amount of parallelization?

- Strong scaling: hold N constant, increase n
- Weak scaling: increase N and n proportionally (with some caveats)

Gustafson's law lets us theoretically calculate the weak scaling speedup. Let S remain constant, while the runtime of P increases with n. Then, we have,

$$\text{Speedup} = \frac{S + Pn}{S + P} = S + Pn \tag{2}$$

Keep in mind that both of these laws are abstractions; reality is much more complicated.

1 Parallel Computing

2 Computer Architectures

3 Models of Parallel Computing

4 Parallel Computing Frameworks

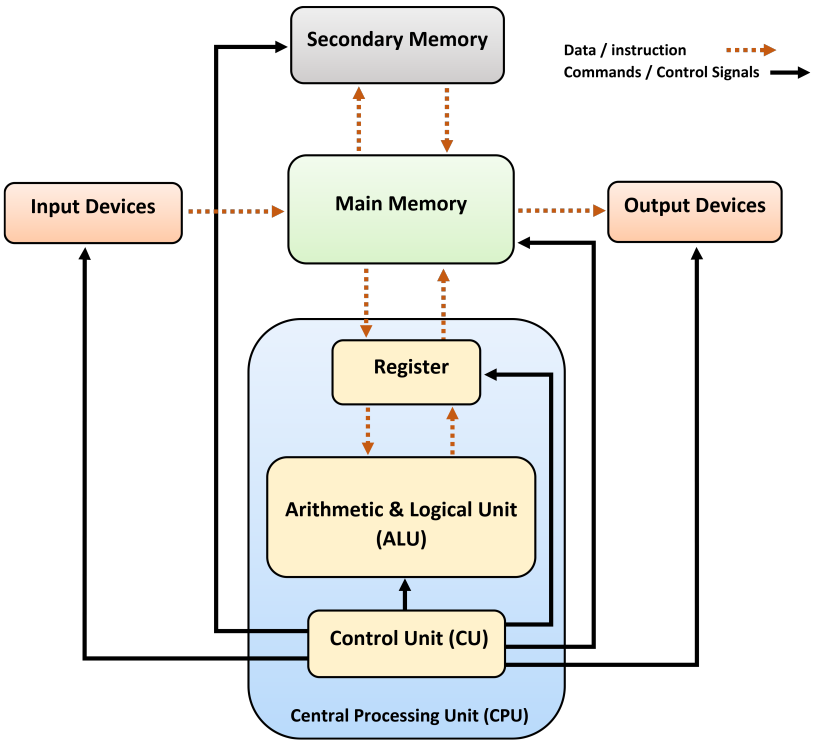
5 MPI

6 Project 3

Computer Architecture

Computers have:

- A Central Processing Unit (CPU)
- Memory (RAM)
- Other things



Flynn's Taxonomy

Flynn's Taxonomy is a rough way of classifying computer architectures. It breaks down systems in four ways:

- Single instruction stream (SI): your computer handles a serial stream of instructions
- Multiple instruction streams (MI): your computer handles multiple streams of instructions simultaneously
- Single data stream (SD): the instruction(s) are operating on a single data stream
- Multiple data streams (MD): the instruction(s) operate on multiple data streams

Combine to get SISD, SIMD, MISD, and MIMD.

Flynn's Taxonomy

Combing the instruction streams and data streams, we get four computer classifications:

- SISD: old CPUs (approximately)
- SIMD: CPUs with vector instructions (AVX), Graphics Processing Units (GPUs)
- MISD: rare
- MIMD: modern CPU architectures, distributed computing systems

For parallel computing, large problem sizes means that SIMD and MIMD are the primary computer architectures at play.

CPUs

CPUs are the backbone of any computer system, and excel in several areas:

- Serial calculations
- Logical control flows
- Code portability

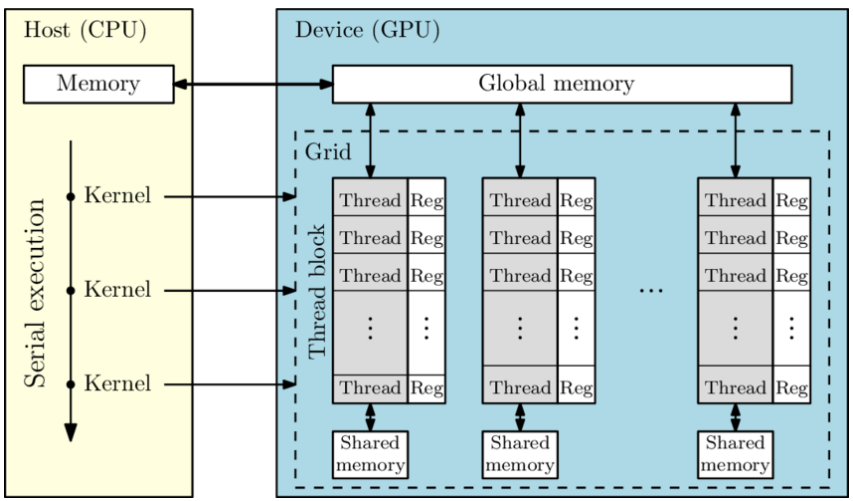
Most modern CPUs have between 2 and 64 cores, so a single CPU already supports some level of parallelism.

GPUs

GPUs are optimized for SIMD calculations where large arrays of floating point numbers are being computed with in some predictable way. GPUs excel at

- SIMD parallelism
- Floating point arithmetic
- Many small tasks

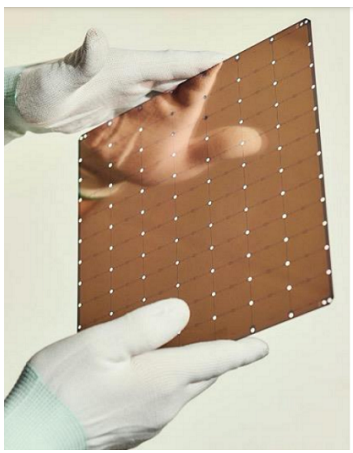
Common GPUs have thousands of cores and can achieve high levels of parallelism, but struggle with logical control statements.



More Exotic Computing Units

GPUs fall under a wider family of computing devices known as accelerators. An accelerator generally refers to specialized hardware that excels at a specific task. Some other accelerators include:

- Systolic arrays
- Field Programmable Gate Arrays (FPGAs)
- Google's Tensor Processing Units (TPUs)
- Apple M-series Matrix Coprocessor
- Cerebras Wafers



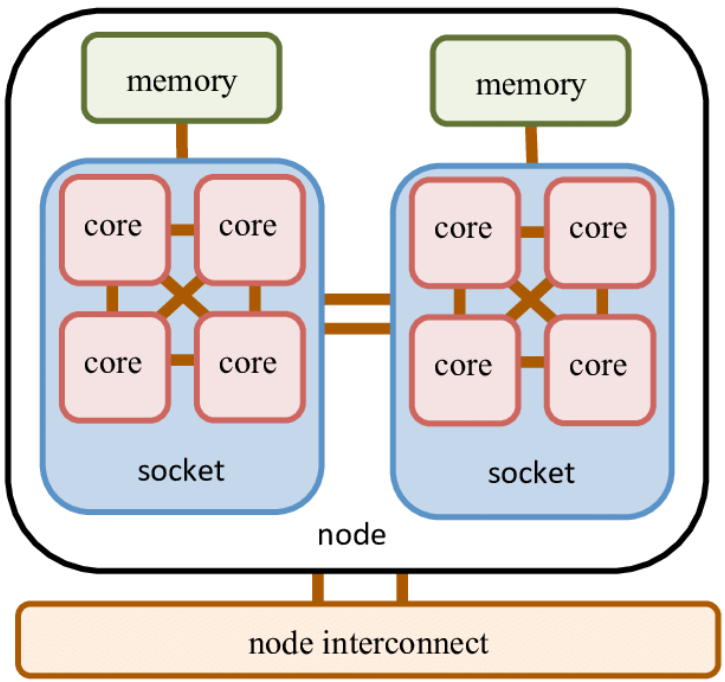
Cerebras Wafer-Scale Engine

	Gen1 WSE	Gen2 WSE
Fabrication process	16 nm	7 nm
Silicon area	46,225 mm²	46,225 mm²
Transistors	1.2 Trillion	2.6 Trillion
AI-optimized cores	400,000	850,000
Memory on-chip	18 GB	40 GB
Memory bandwidth	9 PB/s	20 PB/s
Fabric bandwidth	100 Pb/s	220 Pb/s

A Computing Node

In a high performance computing context, it is common to refer to a node as a unit of a cluster. A node has:

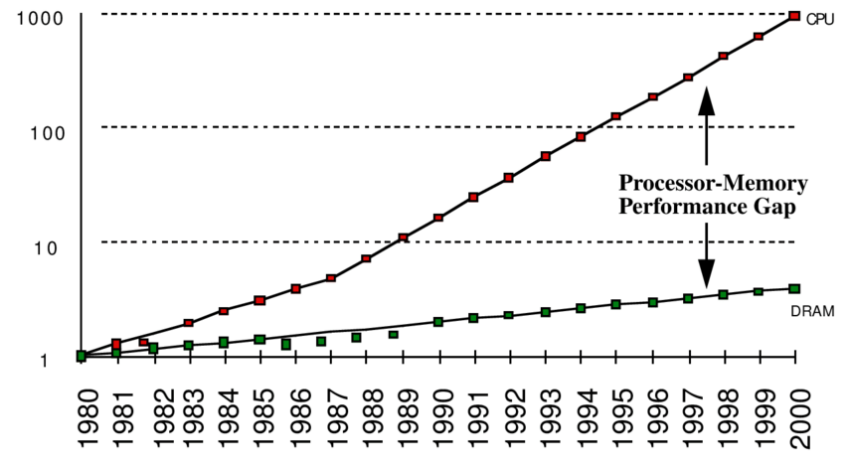
- At least one CPU, frequently more
- A large amount of memory
- Potentially GPUs or other accelerators
- Interconnects with other nodes



On Communication

Communication between various parts of a computer is really really slow. This includes

- Communication between a CPU and Memory
- Communication between CPU nodes
- Communication between a CPU and a GPU



Performance Portability

There are many many different computer architectures. How can you write code that performs well across all types of machines?

1 Parallel Computing

2 Computer Architectures

3 Models of Parallel Computing

4 Parallel Computing Frameworks

5 MPI

6 Project 3

Parallel Programming Models

A model of parallel programming is an abstraction of the underlying computer architecture to help decompose a problem. The main factors to consider in such an abstraction are:

- Process interaction: how can various parallel processes interact with each other?
- Problem decomposition: how is the problem subdivided among various parallel processes?

Shared Memory

This is a simple model of process interaction where many parallel threads operate on a shared pool of memory. Parallelism within a CPU or GPU falls into this category. Advantages include:

- Minimal communication time
- Less memory needed

Disadvantages include:

- Limits on feasible problem sizes
- Race conditions: when multiple threads try to operate on the same data simultaneously

Message Passing

This is another model of process interaction where multiple threads operate independently on individual pools of memory and pass messages when communication is needed. Parallelism across multiple nodes falls into this category. Advantages include:

- Can scale to large problem sizes

Disadvantages include:

- Communication is costly
- Need for synchronization

Task Parallelism

Task parallelism is a method of problem decomposition where different processes perform different tasks. Advantages include:

- Very flexible, applicable to a wide range of algorithms

Disadvantages include:

- Synchronization is likely needed
- Race conditions may occur

Data Parallelism

Data parallelism is where a problem is decomposed by having different threads performing the same operations on different pieces of data. Advantages include:

- Less synchronization delays
- Avoids race conditions

Disadvantages include:

- Less flexible, applicable to less algorithms

Stream Parallelism

Stream parallelism refers to breaking down a pipeline process into different threads. Advantages include:

- Suitable for problems where data is live
- Structured communications are more efficient

Disadvantages include:

- Certain stages may become a bottleneck
- Not suitable for all problems

1 Parallel Computing

2 Computer Architectures

3 Models of Parallel Computing

4 Parallel Computing Frameworks

5 MPI

6 Project 3

MPI: Message Passing Interface

MPI is a standardized and portable message passing standard designed to function on a wide range of architectures.

- MPI is a standard for a message passing interface. MPI is not a specific implementation
- MPI is primarily used in a task and data parallelism context
- MPI includes a variety of types of communication, including point to point, collective, and remote memory access
- MPI is the primary way of performing parallelism across different memory pools

MPI will be discussed a bit more in this lecture, and will be the focus of next week's lecture.

PThreads

pthreads is a standard for the creation and control of threads, independent of a programming language.

- pthreads is an API. Operating systems provide implementations of the API
- pthreads is a form of shared memory parallelism
- p stands for POSIX

OpenMP

OpenMP is another implementation of multithreading, and is based on the use of compiler directives.

- The use of compiler directives means that it is relatively easy to program with
- Implementation is compiler dependent
- Recent iterations of the OpenMP standard have added support for GPUs
- Lots of the parallelization is handled by the compiler, meaning that users have less control

OpenMP will be the focus of lecture 9.

CUDA

CUDA (Compute Unified Device Architecture) is Nvidia's framework for using Nvidia GPUs.

- GPUs are very powerful for the right problems
- Nvidia specific compilers need to be used
- Code portability?
- Writing highly optimized GPU code requires a solid understanding of the GPU hardware architecture

CUDA will be the focus of lecture 10.

Kokkos

Kokkos is a library that aims to handle challenges surrounding performance portability. The idea is that to write code using Kokkos data types, and during the compilation, the parallelization is performed automatically for whatever parallel framework is chosen (threads, OpenMP, CUDA, etc...)

- Maintained by the Linux foundation
- Performance portability is nice
- Necessitates a code rewrite
- Performance probably does not match a direct implementation

Kokkos will be the focus of lecture 11.

1 Parallel Computing

2 Computer Architectures

3 Models of Parallel Computing

4 Parallel Computing Frameworks

5 MPI

6 Project 3

MPI: Message Passing Interface

As previously mentioned, MPI is the standard in the parallel computing world for parallelization across pools of memory.

- MPI is highly optimized and can be competitive with OpenMP for parallelization within a node
- Code is frequently implemented as MPI+X, where MPI is used for parallelization across nodes and another framework is used for intra-node parallelism
- MPI is highly portable
- MPI continues to develop even now

The discussion of MPI in class will focus primarily on the underlying concepts. Many MPI tutorials exist online for more information on coding with MPI.

Implementations of MPI

As previously mentioned, MPI is a standard and not an implementation. Several implementations exist, including:

- MPICH
- Open MPI (not to be confused with OpenMP)
- Intel OneAPI
- HPE

To check if you have MPI installed, you can run `$which mpicxx`

The MPI Programming Model

An independent MPI process is called a rank. MPI ranks can interact in several ways:

- Two MPI ranks can engage in a point to point communication
- A group of MPI ranks belonging to an MPI communicator can engage in a collective communication
- A single MPI rank can engage in a remote memory access communication

Aside from these interactions, MPI ranks act independently.

Sending and Receiving Messages

The main point to point communications in MPI are `MPI_Send` and `MPI_Recv`.

```
int MPI_Send(const void *sendbuf, int count, MPI_Datatype datatype, int dest, int tag,
             MPI_Comm comm)
```

```
int MPI_Recv(void *recvbuf, int count, MPI_Datatype datatype, int source, int tag,
             MPI_Comm comm, MPI_Status *status)
```

MPI Rank `source` sends an array of length `count` of data type `datatype`, located at `*sendbuf` to MPI rank `dest`, which stores the data at `*recvbuf`.

Blocking

The default `MPI_Send` and `MPI_Recv` are blocking communications, which means that they do not return until they are complete. This can pose several problems:

- MPI ranks may spend a lot of time waiting for a matching send or receive to be posted
- Deadlock may occur, where no process can proceed because it is waiting for another process

Both of these are detrimental to parallel efficiency.

Asynchronous Communication

To remedy issues around blocking, `MPI_Send` and `MPI_Recv` have several variants:

- `MPI_Ssend`: returns when the destination rank starts to receive the message
- `MPI_Bsend`: posts a send. If a matching receive is available, the message is passed. If not, the message is first copied into another buffer
- `MPI_Rsend`: a send, assumes a matching receive has already been posted
- `MPI_Isend`: returns immediately, regardless of receiver status. One has to be careful about not modifying the data before the send is complete

1 Parallel Computing

2 Computer Architectures

3 Models of Parallel Computing

4 Parallel Computing Frameworks

5 MPI

6 Project 3

Project 3

For Project 3, your group will create a parallel implementation of an algorithm of your choosing (not necessarily related to data science). Some guidelines:

- You should code in C, or C++ (or Fortran if you want, but please talk to me)
- The parallel implementation should be based on MPI + {OpenMP, CUDA*, Kokkos}
- The simpler of an algorithm you choose to implement, the more sophisticated the parallelization should be

Please get started early!

Deliverables

The final project due date is 13 November. By then, you should submit a report containing:

- An introduction to the algorithm
- A discussion of the design decisions made in parallelization
- A discussion of parallel scaling and efficiency
- A validation that your algorithm is correct
- Detailed runtime figures
- If working with real data, what conclusions you drew about your dataset

Before 30 October, you may choose to submit an optional progress report, detailing what your group has accomplished up to that point. If you do submit the progress report, I will provide feedback and questions to address as some guidance for what else to include in the final report.

Examples

Some data science algorithms you might consider:

- Regression (linear, logistic)
- Tree based algorithms (random forests)
- Clustering algorithms (K-means)
- Neural networks
- Gradient boosting (XGBoost)
- Matrix algorithms (Matrix multiplication, SVD/PCA)
- Dimensionality reduction (t-SNE)

Some datasets which might be useful:

- UC Irvine Machine Learning Repository
- Hugging Face
- Datahub
- Stanford Large Network Dataset Collection

Unsure if an idea you had would be ok? Email me!

HPC Resources

Of course, part of this project is large scale parallelism, which you may not have access to with your personal devices.

Please make sure that you have signed up for HPC access here. There will be further discussion of the HPC resources next lecture, so it would be ideal if you can make an account and access the machines before then.